

STUDY REVIEWER

Generative AI for Software Testing

Based on the ISTQB® Certified Tester – Testing with Generative AI (CT-GenAI) Syllabus v1.1

A 20-page review of all five examinable chapters: key concepts, tables to memorise, exam tips, and a self-check with answers.

How to use this reviewer

- 1. Read** each chapter once, paying attention to the bold terms – every keyword in the syllabus can be tested at K1 (Remember).
- 2. Memorise** the tables (prompt components, prompting techniques, metrics, attack vectors, mitigation lists). These are favourite exam targets.
- 3. Practise** the "Exam angle" boxes: they show how questions tend to be framed (K2 explain/compare, K3 apply/select).
- 4. Test yourself** with the 20 questions at the end, then revisit weak chapters.

Exam at a glance

Item	Detail
Prerequisite	ISTQB® Foundation Level (CTFL) certificate is required before taking the exam.
Format (per exam-prep sources)	40 multiple-choice questions, 60 minutes, pass mark about 65% (26/40). Extra time may apply for non-native speakers. Confirm exact rules with your local Member Board / exam provider.
Knowledge levels	K1 Remember · K2 Understand · K3 Apply. Hands-on objectives (HO), the Introduction and appendices are not directly examinable, but doing the exercises helps K3 questions.
Training time	Minimum 815 minutes (13.6 h) across 5 chapters.
Version note	v1.1 (27 April 2026) is a minor update of v1.0: corrections, terminology tweaks, clarified metrics and risk content, more context on agents. Structure and learning objectives are unchanged.

Study map

Ch.	Title	Time	Weight tip
1	Introduction to Generative AI for Software Testing	100 min	Foundations: tokens, context window, LLM types, chatbot vs LLM-powered tool
2	Prompt Engineering for Effective Software Testing	365 min	Biggest chapter (about 45% of time). Many K3 questions
3	Managing Risks of Generative AI in Software Testing	160 min	Hallucinations, privacy/security, energy, regulations
4	LLM-Powered Test Infrastructure for Software Testing	110 min	Architecture, RAG, agents, fine-tuning, LLMops
5	Deploying and Integrating Generative AI in Test Organizations	80 min	Shadow AI, strategy, model selection, adoption phases, change

Independent study aid based on the ISTQB® CT-GenAI Syllabus v1.1 (© ISTQB®); check the official syllabus for authoritative text.

Chapter 1 · Introduction to Generative AI for Software Testing

Time: 100 minutes · Learning objectives: 1.1.1 (K1), 1.1.2 (K2), 1.1.3 (K2), 1.1.4 (K2), 1.2.1 (K2), 1.2.2 (K2)

Keywords to remember

AI chatbot · context window · deep learning · embedding · feature · foundation LLM · generative AI · generative pre-trained transformer · instruction-tuned LLM · large language model · machine learning · multimodal model · reasoning LLM · symbolic AI · tokenization · transformer

1.1.1 The AI spectrum (K1)

Type	How it works	Testing-related example
Symbolic AI	Rule-based; knowledge represented with symbols and logical rules to mimic human decision-making.	Expert systems, rule engines
Classical machine learning	Data-driven; needs data preparation, feature selection and model training.	Defect categorisation, predicting software problems
Deep learning	Neural networks learn features automatically from very large, complex data (images, audio, text). May still need annotation, tuning and validation by humans.	Image or log pattern recognition
Generative AI	Uses deep learning to create new content (text, images, code) by learning and mimicking patterns in training data.	LLMs writing test cases, scripts, reports

Key advantage for testing: GenAI uses **pre-trained models** that can be applied directly to test tasks with no extra training phase. The trade-off is a set of risks (hallucinations and others, Chapter 3).

1.1.2 Basics of GenAI and LLMs (K2)

- **LLM:** a GenAI model pre-trained on huge text datasets (books, articles, websites); based on the **generative pre-trained transformer** architecture. **SLM** (small language model): compact, fewer parameters, lightweight and focused.
- **Tokenization:** splitting text into **tokens** (a character, sub-word or word). The model processes each token while keeping overall context. Token counts matter for cost, speed and context limits.
- **Embeddings:** numeric vectors in high-dimensional space that encode a token's semantic, syntactic and contextual relationships. **Similar meaning = closer together.**
- **Transformer:** the neural architecture that learns how tokens relate to each other across long sequences. During **inference** the LLM **predicts the next token**.
- **"Plausible is not necessarily correct."** Output is statistically likely text, not verified truth. This single idea explains hallucinations.
- **Non-determinism:** caused by probabilistic inference and hyperparameter settings; the same prompt can yield different outputs.
- **Context window:** the amount of preceding text (in tokens) the model can consider. Bigger window = better coherence over long inputs (e.g. large test logs) but **more computation and longer processing time**.

Exam angle

Expect questions like: "Why might the same prompt give different test cases on two runs?" (probabilistic sampling and hyperparameters) or "What is the drawback of a very large context window?" (compute cost and processing time).

1.1.3 Foundation, instruction-tuned and reasoning LLMs (K2)

Category	Description	Best suited for
Foundation LLM	General-purpose model pre-trained on vast, diverse data (text, code, images...). Powerful and flexible but usually needs further adaptation for specific tasks.	Base for further tuning
Instruction-tuned LLM	Derived from a foundation model, fine-tuned on prompt-response pairs to follow human instructions, stay on task and give coherent answers. Also called "non-reasoning".	Everyday test tasks: drafting, summarising, formatting
Reasoning LLM	Extends instruction-tuned models with structured abilities: logical inference, multi-step problem solving, chain-of-thought. Trained on tasks needing intermediate reasoning steps.	High-cognitive-load tasks: prioritisation with dependencies, estimation, complex analysis

Selection depends on the **complexity and reasoning demands** of the test task (see also 5.1.3).

1.1.4 Multimodal LLMs and vision-language models (K2)

- **Multimodal LLMs** extend the transformer to handle several modalities: text, images, sound, video. Tokenization is adapted per modality; e.g. images are converted into embeddings by a **vision-language model** (VLM) before entering the transformer.
- **Vision-language models** (a subset) combine visual and textual information for image captioning, visual question answering, and checking consistency between text and visuals.
- **Testing uses:** analyse screenshots and GUI wireframes together with user stories or defect reports; spot mismatches between expected and actual visuals; generate test cases that mix textual data and visual cues, improving coverage.

Hands-on (HO-1.1.2, HO-1.1.4): use a tokenizer to see how text splits and count tokens; write a prompt using both text and an image (e.g. a screenshot) for a test task.

1.2 Leveraging GenAI in testing: core principles

1.2.1 Key LLM capabilities for test tasks (K2)

Capability	What the LLM can do
Requirements analysis and improvement	Find ambiguities, inconsistencies, gaps in the test basis; generate clarification questions for stakeholders.
Test case creation support	Draft test cases and suggest test objectives from requirements or user stories.
Test oracle generation	Help produce expected results.
Test data generation	Create datasets, boundary values, combinations.
Test automation support	Generate scripts from test case descriptions; improve existing scripts; suggest techniques.
Test result analysis	Summarise results; classify anomalies by severity and priority.
Testware creation	Draft and keep updated test plans, test reports, defect reports.

1.2.2 AI chatbots vs LLM-powered testing applications (K2)

	AI chatbot	LLM-powered testing application
Interface	Conversational, natural language, direct to the LLM	LLM capabilities integrated via APIs into tools/frameworks

	AI chatbot	LLM-powered testing application
Strengths	Fast feedback, exploratory testing, routine tasks, onboarding, usable by non-technical people; refine with prompt chaining	Customisable, scalable, automates repetitive or complex tasks (test generation, defect analysis, data synthesis)
Advanced form	–	AI agents for specific testing roles (Chapter 4)

Either way, results depend on **strong prompt engineering** (Chapter 2).

Chapter 2 · Prompt Engineering for Effective Software Testing

Time: 365 minutes (largest chapter) · LOs: 2.1.1–2.1.3 (K2), 2.2.1–2.2.5 (K3), 2.3.1–2.3.2 (K2)

Keywords to remember

acceptance criteria · test script · test case · test condition · test data · test design · test report · few-shot prompting · meta prompting · natural language processing · one-shot prompting · prompt · prompt chaining · prompt engineering · system prompt · user prompt · zero-shot prompting

2.1.1 Structure of a prompt: six components (K2)

Component	Purpose	Test example
Role	Perspective or persona the model adopts; sets responsibility and tone.	"Act as a senior test analyst."
Context	Background: test object, functionality under test, relevant circumstances.	"Online banking app; transfer-funds feature."
Instruction	Clear, imperative, concise task description and requirements.	"Generate functional test cases for each acceptance criterion."
Input data	What the model works on: user stories, acceptance criteria, screenshots, code, existing tests, sample outputs.	The user story text and wireframe image.
Constraints	Restrictions or special considerations on how to apply instructions.	"Use boundary value analysis; max 10 cases; no invented rules."
Output format	Expected structure/characteristics of the response.	"Return a table: ID, precondition, steps, expected result."

Memory aid: **R-C-I-I-C-O** (Role, Context, Instruction, Input data, Constraints, Output format). Good structure is combined with a prompting technique.

2.1.2 Core prompting techniques (K2)

Prompt chaining

Break a task into a **series of intermediate steps (multiple prompts)**. Each result is checked (manually or automatically) and refined before the next step. Greater accuracy because each response informs the next prompt; ideal for complicated tasks needing decomposition and systematic checking of intermediate outputs. This is also the natural home of **human verification**.

Few-shot prompting (and its relatives)

- **Zero-shot**: no example; relies on the model's existing knowledge.
- **One-shot**: one example showing the expected result for a given input.
- **Few-shot**: more than one example to consolidate the desired behaviour. Best where examples can show the required pattern; gives **consistent, format-conforming results**.

Meta prompting

Use the AI to **generate or refine its own prompts**. The tester evaluates and refines what the LLM proposes, in an iterative cycle. Helpful when prompt quality and efficiency matter or when the tester is unsure how to write a prompt; it is a form of **pairing** with the GenAI tool (pair testing / pair programming style).

2.1.3 System prompt vs user prompt (K2)

	System prompt	User prompt
Defined by	Developer or tester (set up front)	The chatbot user, every turn
Purpose	Defines behaviour, personality, operational parameters (tone, concision, domain rules, things to avoid); sets the rules for the whole conversation	The actual question, task or instruction for this interaction
Visibility	Usually not visible or editable by the end user	Visible; forms the immediate context
Changes?	Constant during the session	Changes with each interaction
Typical content	Role, context, constraints (and some output guidance)	Specific instruction, input data, output format

The model answers by considering **both together**. Example system prompt: an ISTQB-aligned testing assistant that uses formal language, avoids speculation and cites testing principles.

2.2 Applying prompt engineering to test tasks (K3)

High-quality input is crucial. The quality and relevance of inputs directly affect accuracy and precision of the output. Generated output must always be checked in proportion to risk.

2.2.1 Test analysis with GenAI

Inputs: requirements, user stories, technical specifications, GUI wireframes. Outputs: typical analysis work products such as **prioritised test conditions** (e.g. acceptance criteria).

Task	How GenAI helps
Identify potential defects in the test basis	Spot inconsistencies, ambiguities, incomplete information; compare with similar requirement patterns or past defect reports.
Generate test conditions	Break requirements/user stories into measurable, testable statements.
Prioritise by risk	Using likelihood and impact, regulatory aspects, user-facing features (login, payments), defect history.
Support coverage analysis	Map requirements/stories to conditions; reveal gaps that could become escaped defects.
Suggest test techniques	E.g. boundary value analysis, equivalence partitioning, by requirement type.

Typical chain for refining acceptance criteria (HO-2.2.1b): (1) identify ambiguities → (2) evaluate testability → (3) evaluate completeness, **with human verification and correction at each step** so the next step starts from a clean result. **HO-2.2.1a:** multimodal prompt combining a user story + GUI wireframe to generate acceptance criteria.

2.2.2 Test design and test implementation with GenAI

Task	How GenAI helps
Test case generation	Draft cases from functional and non-functional requirements: preconditions, inputs, expected results, coverage criteria; from simple to end-to-end.
Test data synthesis	Representative, privacy-preserving synthetic data that resembles production, covering extremes, without exposing sensitive information.
Automated test script generation	Turn structured test cases into manual procedures or scripts for various frameworks; update as requirements change.
Scheduling and prioritisation	Analyse cases and interdependencies; optimise execution order by priority, risk, resources, objectives.

- **HO-2.2.2a (prompt chaining + structured + meta prompting):** step 1 generate functional test cases from acceptance criteria in a set format; step 2 verify completeness with a **coverage table** (each criterion covered?); step 3 use a **meta-prompt** to build end-to-end test procedures.
- **HO-2.2.2b (few-shot):** give n examples of user story → test conditions → **Gherkin (Given-When-Then)** scenarios; apply to a new story; refine prompt or examples if inaccurate.
- **HO-2.2.2c (chaining):** prioritise test cases using priorities and dependencies (risk-, coverage-, requirements-based). A multi-criteria reasoning task, so **manually verify** the LLM's reasoning.

2.2.3 Automated regression testing with GenAI

Regression suites grow every iteration, so they suit automation, especially in **CI/CD** pipelines. GenAI assists creation, maintenance and optimisation of the suites.

Activity	How GenAI helps
Keyword-driven script implementation	Map keywords (predefined, common test steps) to test cases and generate scripts.
Impact analysis and test optimisation	Analyse code changes to find high-risk areas and focus regression effort.
Self-healing and adaptive tests	Adjust scripts automatically to minor UI/API changes (dynamic locators, changed interactions, request/response formats, endpoints, authentication).
Automated test reporting and insights	Detailed, timely reports with metrics, failures, trends and predictive insights.
Enhanced defect reporting and root cause analysis	Compile defect reports with logs, screenshots and environment data.

GUI tests are unstable because of frequent UI change; **API tests** suffer from changing formats, endpoints and authentication. GenAI can adapt scripts and generate diverse data for both. **HO-2.2.3a:** few-shot prompting to create/manage keyword-driven scripts, plus a system-prompt-based AI assistant that checks and corrects scripts. **HO-2.2.3b:** structured prompts for report analysis: analyse results vs specification → cluster similar defects → maintain a known-anomalies list → cross-check, all in one conversation.

2.2.4 Test monitoring and test control with GenAI

- **Monitoring and metrics analysis:** automate monitoring, analyse trends, predict risks, alert on deviations from plan.
- **Test control:** insights for reprioritising tests, adjusting schedules, reallocating resources.
- **Test completion insights and continuous learning:** completion reports, successes and lessons learned.
- **Visualisation and reporting:** dynamic dashboards and natural-language summaries for all stakeholders.

2.2.5 Choosing prompting techniques (K3) – memorise this table

Technique	Recommended use case	Typical applications
Prompt chaining	Complex tasks needing precision with human verification at each step	Test analysis, test design, test automation where each step is checked for correctness
Few-shot prompting	Repetitive tasks or tasks with a specific/constrained output format	Gherkin scenarios, keyword-driven testing, reports with a fixed format
Meta prompting	Flexible, dynamic tasks; crafting prompts for new tasks	Complex tasks such as test report analysis and anomaly detection

Techniques can be **combined**: meta prompting produces an initial prompt → it contains examples you adapt (few-shot) → split into subtasks to validate intermediate results (chaining).

Exam angle (K3 selection questions)

"Output must always follow this exact Gherkin layout across 200 stories" → **few-shot**.

"The task is complex and each intermediate result must be validated by a tester" → **prompt chaining**.

"The tester does not know how to phrase a prompt for a new kind of task" → **meta prompting**.

If a scenario has both a repeated format **and** a complex flow, expect a **combination** to be the best answer.

2.3 Evaluating results and refining prompts (K2)

2.3.1 Metrics for evaluating GenAI results on test tasks

Metric	Description	Example
Accuracy	Overall correctness vs expert-written tests, requirements or standards	Degree generated tests cover all specified requirements
Precision	Correctness with respect to a specific objective	Generated tests correctly identify anomalies
Recall	Ability to find all relevant instances	Tests cover valid and invalid equivalence partitions of a data class
Relevance and contextual fit	Applicable and appropriate for the context	Consistent with the test basis; domain rules integrated
Diversity	Wide range of inputs/scenarios, avoiding repetition	Various user behaviours and edge cases explored
Execution success rate	Share of generated artefacts executable as is	Scripts run without syntax or format errors
Time efficiency	Time saved vs manual effort	Generation time vs human time for equivalent tests

Task-specific metrics can be added. Evaluate by manual review or automatically against a reference. Because of non-determinism, metrics must rest on **statistically relevant data** (many runs, not one).

2.3.2 Techniques for evaluating and refining prompts

Technique	Idea
Iterative prompt modification	Start with a base prompt; adjust wording/terminology; add context step by step.
A/B testing of prompts	Compare versions on predefined metrics to see which structure or phrasing works best.
Output analysis	Study errors/inconsistencies (e.g. vs the test basis) and change the prompt to avoid them.
Integrate user feedback	Collect testers' views on usefulness, clarity, level of detail; feed into prompt refinement.
Adjust length and specificity	Sometimes more context helps; sometimes shorter prompts generalise better.

Share and standardise through **prompt libraries** and team evaluation sessions to build a culture of continuous improvement.

2.4 Worked prompt examples (illustrative study aids)

These examples are written for revision; they are not quoted from the syllabus. Use them to practise **identifying components and techniques**, a typical K2/K3 skill.

Example A: a structured prompt for test analysis

Component	Content
Role	You are an experienced test analyst.
Context	We are testing the "Reset password" feature of a mobile banking app.
Instruction	Identify ambiguities and missing information in the user story below, then list the questions we should ask the product owner.
Input data	"As a customer I want to reset my password so that I can regain access." Attached: wireframe screenshot.
Constraints	Do not invent business rules. Mark every assumption explicitly. Maximum 8 questions.
Output format	Numbered table: Ambiguity Why it matters Question for stakeholder.

Example B: prompt chaining for acceptance criteria

1. **Prompt 1:** "List ambiguities in this user story." → tester reviews and corrects the list.
2. **Prompt 2:** "Using the corrected list, rewrite the acceptance criteria so each is testable." → tester checks testability.
3. **Prompt 3:** "Check the criteria for completeness against the wireframe and list any gaps." → tester verifies; gaps are fed back.

Example C: few-shot prompt for Gherkin

The prompt contains **two or three complete examples** (user story → test conditions → Given/When/Then scenarios) followed by a **new user story** with the instruction "produce scenarios in exactly the same style". If output drifts, refine the examples, not just the wording.

Example D: meta prompt

"I need a prompt that makes an LLM analyse regression test reports and cluster similar failures. Ask me any questions you need, then write the prompt using role, context, instruction, input data, constraints and output format." The tester evaluates the generated prompt, tests it on sample data and refines it.

Practice: which technique is each?

1. A tester pastes three finished keyword-driven scripts and asks for a fourth in the same style → **few-shot**.
2. A tester asks the LLM to draft the prompt that will later be used for anomaly detection → **meta prompting**.
3. A tester asks for conditions, reviews them, then asks for test cases from the approved conditions → **prompt chaining**.
4. A tester asks for boundary tests with no examples → **zero-shot** (a structured prompt, but no technique from the "core three").

Chapter 3 · Managing Risks of Generative AI in Software Testing

Time: 160 minutes · LOs: 3.1.1 (K1), 3.1.2 (K3), 3.1.3 (K2), 3.1.4 (K1), 3.2.1–3.2.3 (K2), 3.3.1 (K2), 3.4.1 (K1)

Keywords to remember

security · vulnerability · data privacy · hallucination · temperature · reasoning error · bias · context manipulation

3.1 Hallucinations, reasoning errors and biases

3.1.1 Definitions (K1)

Defect	Definition	Test-task example
Hallucination	Output that appears factually incorrect or irrelevant to the task.	Fictitious test cases, non-working scripts, tests for non-existent acceptance criteria
Reasoning error	Misinterpreting logical structures (cause and effect, conditional logic, step-by-step processes) leading to wrong conclusions. LLMs rely on pattern matching, not true logical reasoning.	Test planning, effort estimation, test case prioritisation
Bias	Output favouring certain information, approaches or assumptions, originating from training data.	English-centric data underrepresents other perspectives; skewed test data or acceptance criteria

All three stem from training data and inherent limits of the transformer model. Because of **non-determinism**, a defect that seems fixed in one output can **reappear** in another conversation.

3.1.2 Identifying them in LLM output (K3)

Problem	Detection approaches
Hallucination	Cross-verification against documentation, requirements, known behaviour · Domain expertise consultation (SMEs) · Consistency checks between outputs and known information
Reasoning error	Logical validation of the reasoning flow (review cycles, human judgement for complex cases) · Output testing: run the generated tests/scripts against the test object
Bias	Review whether testware (test code, synthetic data) fairly represents the test strategy and coverage requirements · Look for underrepresented test types (e.g. missing non-functional tests)

How much detection effort to spend depends on the **estimated risk level** of the task.

3.1.3 Mitigation (K2)

- **Provide complete context** (all six prompt components).
- **Divide prompts into manageable segments** (prompt chaining) and verify each output.
- **Use clear, interpretable data formats**; avoid ambiguous ones.
- **Select the appropriate model** for the task.
- **Compare results across models** to expose errors and choose the most reliable output.
- Chapter 4 adds two complementary techniques: **RAG** and **fine-tuning**.

3.1.4 Mitigating non-deterministic behaviour (K1)

Strategy	Effect and caveat
Lower the temperature	Narrows the probability distribution → less randomness, more consistent. Caveat: less creativity/diversity; outputs more repetitive.

Strategy	Effect and caveat
Set a random seed	Where supported, the same pseudo-random sequence is reused → better reproducibility.
Automate parts of output verification	Structured, consistent evaluation of results.

Full reproducibility **cannot be guaranteed**, especially for long outputs.

Exam angle

Remember the direction: **higher temperature = more varied/creative; lower = more deterministic**. A question may ask which setting best supports reproducible regression-oriented generation (low temperature, fixed seed) and what you sacrifice (diversity).

3.2 Data privacy and security risks

3.2.1 Key risks (K2)

Privacy

- **Unintentional data exposure:** outputs may reveal sensitive information.
- **Lack of control over data usage:** tools may store/process sensitive data without explicit consent or control.
- **Compliance risks:** e.g. **GDPR** (Regulation (EU) 2016/679); GDPR does not ban GenAI but its safeguards (lawfulness, purpose limitation, storage limits) constrain what you may do.

Security

- LLM-powered test infrastructure can suffer breaches or unauthorised access.
- Attackers can exploit LLM weaknesses to alter behaviour or extract sensitive data, or feed malicious input to mislead the model.

3.2.2 Attack vectors (K2) – memorise

Attack vector	Description	Example in testing
Context manipulation	Requests designed to extract confidential training data	Overloading the context window with long prompts so the model reveals random snippets of training data
Request manipulation	Introducing data that disrupts the AI's output	Images that lure the AI into another context, causing hallucinated acceptance criteria
Data poisoning	Manipulating training data	Fake evaluations when rating an AI-generated test report
Malicious code generation	Manipulating an LLM into generating backdoors	Generated code that opens a channel to a malicious IP

3.2.3 Mitigation strategies (K2)

Data-privacy measures	Additional strategies
Data minimisation: avoid sensitive data unless legally permitted; use only what is needed	Systematic human review of output
Anonymisation / pseudonymisation: mask or replace sensitive data	Cross-check with another LLM
Secure storage and transmission: encryption, access controls	Choose a secure environment by confidentiality need: commercial secure offering, secure cloud, or on-premises model
Resource training: policies and training for responsible use	Regular security audits and vulnerability assessments
	Stay current with security best practice

Strategies are complementary; combine them. Involve **senior security engineers, legal counsel, the CTO or CISO** where they exist.

3.3 Energy consumption and environmental impact (K2)

- Training and running LLMs needs intensive specialised compute; web-based use loads devices, networks and data centres.
- Energy use **risks sharply with usage**, and depends on **task complexity and computational resources**. Generating one image with a powerful model can use about as much energy as fully charging a smartphone, whereas text generation uses a small fraction.
- Accurate figures are hard to obtain, but cumulative use across millions of users means significant **CO₂ emissions**.
- **Best practice:** limit unnecessary model interactions (concise prompts, right-sized model, avoid pointless regeneration).
- **HO-3.3.1:** use a simulator to estimate energy and CO₂ for given GenAI test tasks.

3.4 AI regulations, standards and frameworks (K1)

Name	Type	What it is / relevance to GenAI in testing
ISO/IEC 42001:2023	Standard	AI management system requirements for organisations; supports recommended practice, consistency and reliability.
ISO/IEC 23053:2022	Standard	Framework for AI systems using machine learning; AI lifecycle processes with emphasis on safety and transparency; data quality, transparency, safety.
EU AI Act	Regulation	Legal framework addressing AI risks, classifying applications by risk level; transparency, accountability, bias mitigation.
NIST AI RMF 1.0	Framework (US)	Guidelines for managing AI risks: fairness, transparency, security; helps prevent biased test results.

Test organisations must keep up with evolving regulations, standards, national laws and frameworks.

Quick self-check for Chapter 3

Can you (a) define the three output defects, (b) list two detection methods for each, (c) name the four attack vectors with an example, (d) list five privacy/security mitigations, (e) match the four regulations/standards to their type? If yes, you are ready for the K1/K2 items.

Worked scenarios: diagnose and mitigate

Scenario	Diagnosis	Best mitigation
The LLM produces a test for a "biometric login" that appears nowhere in the requirements.	Hallucination	Cross-verify against requirements; provide complete context; consider RAG grounding; human review.
Given 12 test cases with dependencies, the LLM schedules a case before its prerequisite.	Reasoning error	Prompt chaining with verification per step; use a reasoning model; run/validate the schedule.
All generated test data uses names and addresses from one region; no accessibility tests appear.	Bias (data and test-type coverage)	Review vs test strategy; ask explicitly for diverse data and non-functional tests; compare models.
A tester pastes production customer records into a public chatbot to "get realistic data".	Privacy and compliance risk (also shadow AI if unapproved)	Data minimisation, anonymisation or synthetic data, approved secure environment, policy and training.

Scenario	Diagnosis	Best mitigation
Running the same prompt daily gives differently structured regression reports.	Non-determinism	Lower temperature, set seed, fixed output format (few-shot), automated verification.
Users rate an AI-generated test report in a shared tool; a bot posts fake ratings that steer future output.	Data poisoning	Access controls, input validation, audit and monitoring of feedback sources.

Chapter 4 · LLM-Powered Test Infrastructure for Software Testing

Time: 110 minutes · LOs: 4.1.1, 4.1.2, 4.1.3, 4.2.1, 4.2.2 (all K2)

Keywords to remember

test infrastructure · fine-tuning · LLM-powered agent · Large Language Model Operations (LLMOps) · retrieval-augmented generation (RAG) · vector database

4.1.1 Architecture of an LLM-powered test infrastructure (K2)

Component	Role
Front-end	User interface where testers enter queries or commands.
Back-end	Handles authentication, data retrieval, prompt preparation, interaction with the LLM, and post-processing of raw output so it fits the test process before returning it to the front-end.
LLM	Third-party service (via API) or custom in-house model; generates responses from structured prompts.
Data sources	Relational databases (structured data such as test cases) and vector databases (semantic retrieval using embeddings).

It goes beyond client-server: the LLM is a **smart processing component** that reasons over testware, and insights are generated **dynamically from context** (requirements, code, results), unlike scripted rule-based chatbots.

4.1.2 Retrieval-Augmented Generation (K2)

RAG improves relevance and accuracy by adding **external data sources** to response generation, giving **grounded** answers.

Phase	Steps
Preprocessing (indexing)	Split large documents into chunks (e.g. 256–512 tokens) → clean and process → encode each chunk as an embedding → store in a vector database.
Runtime step 1: Retrieval	Encode the user query; retrieve chunks by semantic similarity to the query embedding.
Runtime step 2: Generation	Feed retrieved chunks to the LLM as context; it combines existing knowledge with the new data to produce a grounded response.

In testing: lets test tools tap enterprise data (databases, documentation, repositories) in real time so analysis and design align with the latest specifications, requirements and existing tests. **HO-4.1.2:** compare LLM output with and without RAG. Example: a banking test case for one function, using specs from a vector database and existing tests from a relational database.

4.1.3 LLM-powered agents (K2)

- Specialised GenAI applications for **semi-autonomous or autonomous** processing of defined tasks. Unlike chatbots, they can **act** by invoking predefined functions called **tools**, manipulating external systems.
- **Autonomous agents:** minimal human intervention. **Semi-autonomous agents:** periodic human oversight.
- **Multi-agent architectures:** several specialised agents communicate and coordinate ("**orchestration**") to solve complex problems.
- Exposed to users as AI assistants in test tools; can transform user stories/requirements into artefacts across analysis, design, implementation, execution and reporting, shifting from **script-based** to **goal-driven, agent-based** automation.
- **Risk:** agents inherit hallucinations, reasoning errors and biases. **Mitigate** with automated verification of agent results, and use **semi-autonomous agents for critical tasks**.

4.2 Fine-tuning and LLMOps

4.2.1 Fine-tuning (K2)

Adapt a pre-trained LM (LLM or SLM) to specific tasks or domains by **further training on a targeted dataset**, so it learns domain knowledge and vocabulary. Suited to giving generic models **specialised reasoning** or a field-specific vocabulary. Compared with RAG (which supplies knowledge at query time), fine-tuning changes the **model itself**, needing curated data, compute and re-tuning when needs change.

4.2.2 LLMOps (K2)

Large Language Model Operations covers deploying, monitoring and maintaining LLMs in production, the operational pipeline around the model. For testing, three deployment approaches are compared, each with **data privacy and cost** implications:

Approach	Notes
AI chatbots	Quickest to adopt; least control over data handling; provider-managed.
GenAI test tools (commercial LLM-powered tools)	Vendor-integrated; check vendor security, data use terms and licensing.
In-house GenAI tools	Greatest control and customisation (including hosting and fine-tuning); highest build and operations effort and cost.

Note: I condensed 4.2 from the official summaries; re-read section 4.2 of the syllabus for the exact LLMOps wording.

Chapter 5 • Deploying and Integrating GenAI in Test Organizations

Time: 80 minutes • covers 5.1 Roadmap for adoption and 5.2 Change management

5.1.1 Risks of shadow AI

Shadow AI = staff using **unapproved AI tools** without clear guidelines. Risks: **data privacy and information security**, regulatory and **intellectual-property** issues (e.g. GDPR), possible legal consequences. Mitigate with approved tools, clear policies, training and monitoring.

5.1.2 Key aspects of a GenAI strategy in testing

- Define **measurable testing objectives** (e.g. reduce manual test-case writing time).
- Select **compatible LLMs/SLMs**.
- Ensure **data quality, training, measurement and compliance**; consider costs and a roadmap.

5.1.3 Selecting LLMs/SLMs for test tasks

Match model **capabilities, supported inputs, reasoning ability and outputs** to the task, and weigh **benchmarks and evaluation results, licensing, cost, hosting options, performance and fine-tuning potential**. Run **pilots** to compare models.

5.1.4 Phases of adoption

Phase	Focus
1. Discovery	Understand needs, opportunities and risks; define test objectives and plan.
2. Initiation	Run pilots; compare models; check data quality and compliance; measure benefits.
3. Usage definition	Define how GenAI is used in day-to-day test processes; scale with guidelines.

5.2 Managing change

- **Essential skills and knowledge (5.2.1)**: understanding LLM capabilities and limits, prompt engineering, critical evaluation of output (hallucination/bias awareness), data privacy and security awareness, and solid core testing knowledge.
- **Building capabilities (5.2.2)**: hands-on, **incremental learning**; training; shared prompt libraries; communities of practice.
- **Evolving test processes (5.2.3)**: add review and verification steps for AI output, define roles/responsibilities and update processes as tools mature.

Note: 5.2 is summarised at a high level; check the syllabus for its exact lists.

Adoption checklist (illustrative synthesis)

The list below turns the syllabus themes into a practical checklist; it is a study aid, not official syllabus text.

Theme	Questions a test organisation should answer
Objectives	Which test tasks will GenAI support? What measurable target (time saved, coverage, defects found) defines success?
Model choice	Do capabilities, input types, reasoning level, licensing, cost, hosting and performance fit? Was a pilot comparing several models done?
Data and privacy	What data may be sent to the model? Is anonymisation or synthetic data required? Which environment is approved (commercial secure offering, secure cloud, on-premises)?

Theme	Questions a test organisation should answer
Risk and quality	How will output be verified (human review, cross-model comparison, automated checks)? Which metrics are tracked, on enough runs to be statistically meaningful?
Governance	Which regulations and standards apply (GDPR, EU AI Act, ISO/IEC 42001, NIST AI RMF)? Who owns approval: security, legal, CTO/CISO?
People	What training exists? Are prompt libraries shared? Are roles and processes updated for AI-assisted work?
Sustainability	Are unnecessary model calls avoided and model size matched to the task?

Exam angle for Chapter 5

Questions usually ask you to **recognise shadow AI risks**, **choose what belongs in a strategy** (measurable objectives, model selection, data quality, training, compliance) or **order the adoption phases** (discovery → initiation → usage definition).

Cheat sheet: commonly confused pairs

Pair	Difference in one line
Foundation vs instruction-tuned LLM	Foundation = general pre-trained base; instruction-tuned = tuned on prompt-response pairs to follow instructions.
Instruction-tuned vs reasoning LLM	Reasoning models add multi-step inference and chain-of-thought for complex logic.
LLM vs SLM	SLM has fewer parameters: lightweight, focused.
AI chatbot vs LLM-powered test tool	Chatbot: conversational, direct; tool: API-integrated, automated, scalable.
LLM-powered agent vs chatbot	Agent invokes tools and acts; chatbot answers questions.
Autonomous vs semi-autonomous agent	Semi-autonomous has periodic human oversight; preferred for critical tasks.
System vs user prompt	System = fixed behaviour rules set up front; user = the per-turn request.
Zero- vs one- vs few-shot	No example / one example / several examples.
Prompt chaining vs meta prompting	Chaining splits a task into verified steps; meta prompting has the LLM craft prompts.
Precision vs recall	Precision = correctness of what was produced; recall = how much of the relevant was found.
Accuracy vs execution success rate	Accuracy = correctness vs reference; execution success = runs as is without errors.
RAG vs fine-tuning	RAG supplies external context at query time; fine-tuning further trains the model itself.
Hallucination vs reasoning error	Hallucination = false or irrelevant content; reasoning error = faulty logic.
Anonymisation vs pseudonymisation	Anonymisation removes identifiability; pseudonymisation replaces identifiers with substitutes (both are masking approaches).
Temperature vs seed	Temperature controls randomness of sampling; seed fixes the pseudo-random sequence.
ISO/IEC 42001 vs 23053	42001 = AI management system requirements; 23053 = framework for ML-based AI systems.

Numbers and lists to memorise

- Chapter times: **100 / 365 / 160 / 110 / 80** minutes (total 815).
- **6** prompt components · **3** core prompting techniques · **7** evaluation metrics · **5** prompt-refinement techniques.
- **4** attack vectors · **4** regulations/standards/frameworks in 3.4 · RAG chunk example **256–512 tokens**.
- **3** adoption phases: discovery, initiation, usage definition.
- Cognitive levels: **K1** remember, **K2** understand, **K3** apply (K3 items: 2.2.1–2.2.5 and 3.1.2).

Master glossary of GenAI terms

Term	Meaning (short)
AI chatbot	Conversational interface to an LLM.
Bias	Skewed output rooted in training data.
Context window	Tokens of preceding text the model can consider at once.
Context manipulation	Attack using crafted/overlong input to extract data.
Embedding	Numeric vector capturing meaning and relationships of a token/chunk.
Few-shot / one-shot / zero-shot	Prompt with several / one / no examples.
Fine-tuning	Further training on a targeted dataset for a task or domain.
Foundation LLM	General-purpose pre-trained model needing adaptation.
Hallucination	Factually wrong or irrelevant output.
Instruction-tuned LLM	Foundation model tuned on prompt-response pairs.
LLM-powered agent	LLM that can call tools and act semi-/autonomously.
LLMOps	Operations for deploying and managing LLMs.
Meta prompting	Using the LLM to create or refine prompts.
Multimodal LLM / VLM	Handles several data types / text plus vision.
Prompt chaining	Sequence of verified prompts breaking a task into steps.
RAG	Retrieval of relevant chunks to ground the LLM response.
Reasoning LLM	Model strengthened for multi-step logical inference.
Reasoning error	Faulty logic or conclusions.
SLM	Small language model: compact, focused.
Shadow AI	Unapproved AI tool use in an organisation.
System / user prompt	Persistent behaviour rules / per-turn request.
Temperature	Sampling setting: lower = more deterministic.
Tokenization	Splitting text into tokens.
Vector database	Store enabling similarity search on embeddings.

Self-check: 20 questions with answers

#	Question	Answer
1	Which AI type learns features automatically using neural networks?	Deep learning.
2	Which LLM category is best for multi-step prioritisation with dependencies?	Reasoning LLM.
3	What does a context window measure?	The amount of preceding text (in tokens) the model can consider.
4	Name the six prompt components.	Role, context, instruction, input data, constraints, output format.
5	Which technique fits a fixed Gherkin output across many stories?	Few-shot prompting.
6	Which technique needs human verification at each step?	Prompt chaining.
7	Who typically defines the system prompt?	The developer or tester; end users usually cannot see or edit it.
8	What does the execution success rate metric measure?	Share of generated artefacts that run as is in the test environment.
9	Why must metrics use statistically relevant data?	Because LLM output is non-deterministic.
10	Give two ways to reduce non-determinism.	Lower temperature; set a random seed.
11	Which task is a typical source of reasoning errors?	Test case prioritisation or effort estimation.
12	Which detection method runs generated scripts against the test object?	Output testing (reasoning error detection).
13	Which attack vector is "fake evaluations of an AI test report"?	Data poisoning.
14	What is data minimisation?	Using only the necessary, non-sensitive data with GenAI.
15	What increases sharply with GenAI use and affects CO ₂ ?	Energy consumption.
16	Which regulation classifies AI applications by risk level?	EU AI Act.
17	What are the two RAG runtime steps?	Retrieval, then generation.
18	How do agents differ from chatbots?	Agents can invoke tools to act on external systems.
19	What is shadow AI?	Use of unapproved AI tools without guidelines.
20	Name the three adoption phases.	Discovery, initiation, usage definition.

Good luck with your CT-GenAI exam! Re-check any weak topic directly in the official syllabus.